

Bayesian Networks assignment

Alessandro Mancuso
Radboud University

s1148451
April 2026

1 Introduction

Bayesian Networks are probabilistic graphical models that represent a set of variables and their dependency relations through a directed acyclic graph. Each node corresponds to a variable, while each directed edge encodes an influence relationship: a parent node directly affects the probability distribution of its child [2].

In addition to the graphical structure, a Bayesian Network includes a Conditional Probability Table (CPT) for every node. Each CPT specifies the probability distribution of a variable for every possible joint value assignment of its parents.

Bayesian Networks provide a compact factorization of the joint probability distribution over all variables:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

This factorization represents the conditional independence relations encoded in the graph, enabling the computation of marginal and conditional probabilities.

A key computational tool in Bayesian Networks are the factors. A factor can be seen as a multidimensional table whose dimensions correspond to the variables it involves. Factors offer an efficient way to represent CPTs and form the fundamental data structure required to perform inference algorithms such as Variable Elimination.

Variable Elimination (VE) is an exact inference algorithm that computes marginal probabilities by eliminating non-query and non-evidence variables. It proceeds by conditioning factors on the observations, multiplying the resulting factors and marginalizing variables according to a chosen elimination order. VE can be used to obtain the normalized probability distribution through marginalization, or as the computational basis for Maximum a Posteriori (MAP) inference, where marginalization is replaced by maximization over the query variables.

In this assignment, we represent Bayesian Networks using factors and develop the fundamental operations required to perform computations over them. Building on this representation, we then implement the Variable Elimination algorithm to compute exact marginal and posterior probabilities. Finally, we apply the MAP algorithm to determine the most probable assignment of the query variables given the observed evidence.

In the last part of the assignment we cover the Expectation-Maximization algorithm (EM), an iterative optimization technique used to estimate unknown parameters in probabilistic models such as Bayesian Networks. This tool is particularly useful when a dataset is incomplete, noisy or contains hidden variables. In this setting, we develop the EM algorithm to learn the conditional probability tables (CPTs) of the *Endorisk Bayesian Network* from a dataset generated by the same model, where some variables are treated as hidden. To observe the behavior of the algorithm we perform multiple runs using several different random initialization of the CPTs involving hidden variables. This allows us to analyze the convergence properties of EM and assess its sensitivity to initialization. Finally, the learned parameters are compared with those of the original Endorisk model.

2 Methods

2.1 Project setup and codebase

This project is implemented in Python and builds upon a provided codebase that handles the parsing, representation, and initialization of a Bayesian Network from a `.bif` file. On top of this foundation, we develop the full set of methods required to work with factors and implement the Variable Elimination algorithm used to calculate marginal probability distribution and most probable assignment and to perform expectation-maximization.

The given codebase is structured as follows:

```
project_root/  
|-- run.py  
|-- read_bayesnet.py  
|-- variable_elim.py  
|-- earthquake.bif
```

The `run.py` file contains the entry point of the Python application. It initializes the Bayesian Network by loading a `.bif` file, constructs an instance of the Variable Elimination class, and provides a template for executing inference queries.

The `read_bayesnet.py` file provides all the functionality required to parse and represent a Bayesian Network from a `.bif` file. It extracts the set of variables,

their possible values, the parent relationships, and the associated Conditional Probability Tables (CPTs), storing them in data structures based on **pandas** DataFrames. The parser was extended to support variable names containing spaces. Additionally, the probability parsing was also made more robust by skipping empty or malformed lines within probability blocks, preventing runtime errors when processing more complex .bif files.

The **earthquake.bif** file is an example Bayesian Network distributed with the assignment. It encodes a “Burglary–Earthquake–Alarm–JohnCalls–MaryCalls” model, including its variables, their domains, and all conditional probability distributions.

The **variable_elim.py** file contains the skeleton of the **VariableElimination** class. This class is initially empty and meant to be fully implemented as part of the assignment. It provides the structure into which the factor operations and the complete Variable Elimination algorithm will be developed.

2.2 Implementation

After implementing the software covered by the assignment, the updated directory tree looks as follows:

```
project_root/
|-- data
|   |-- simulation_data_hid_names.dat
|   |-- simulation_data_hid.dat
|
|-- networks
|   |-- earthquake.bif
|   |-- endomcancer.bif
|   |-- endorisk.bif
|   |-- endorisk_new.bif
|
|-- logs
|   |-- variable_elim_logs.txt
|   |-- map_logs.txt
|   |-- expectation_maximization_logs.txt
|
|-- plots
|   |-- em_convergence.png
|
|-- read_bayesnet.py
|-- factor.py
|-- heuristics.py
|-- pgmpy_utils.py
|-- variable_elim.py
```

```
|-- expectation_maximization.py
|-- run.py
|-- run_em.py
|-- README.md
```

The implementations carried out will be explained below.

Firstly, the **run.py** file has been updated as shown in Figure 1. This file represents the entry point of the inference part of the assignment and handles the execution of the Variable Elimination algorithm. At the beginning of the execution, the user is prompted to select whether to run the program in debug mode. When enabled, this mode produces additional logs that provide some logs about the intermediate steps performed by the algorithm, which is useful for understanding and verifying the computation process.

```
101 if __name__ == '__main__':
102     # The class Bayesian represents a Bayesian network from a .bif file in several variables
103     base_path = os.path.dirname(__file__)
104     bn_path = os.path.join(base_path, 'networks', 'earthquake.bif')
105     bn_networks_path = os.path.join(base_path, 'networks')
106     selected_network = network_selection(bn_networks_path)
107     selected_bn_path = os.path.join(bn_networks_path, selected_network)
108     net = Bayesnet(os.path.join(base_path, 'networks', selected_network))
109     debug_mode = bool(input_user("Do you want to run the code in debug mode (yes/no): "))
110     if debug_mode:
111         # Print network if debug mode is selected
112         print_network(net)
113     # Initialize variable elimination data structures
114     ve = VariableElimination(net, debug_mode)
115     # Algorithm selection
116     algorithm_choice = algorithm_selection()
117     # Query and evidence selection
118     query, query_vars, evidence = query_and_evidence_selection(net, algorithm_choice)
119     if debug_mode:
120         print("-----")
121         if algorithm_choice == "VE":
122             print("RUN ALGORITHM VARIABLE ELIMINATION MARGINAL")
123         else:
124             print("RUN ALGORITHM VARIABLE ELIMINATION MOST PROBABLE ASSIGNMENT (MAP)")
125         print("MAP QUERY VARIABLES:", query_vars)
126         print("EVIDENCE:", evidence)
127         print("-----")
```

(a) run.py module - 1

```
128 ve.heuristic = ve.heuristic_selection()
129 # Heuristic selection
130 heuristic = [
131     "1", eliminate_heuristic_simple_sorting,
132     "2", eliminate_heuristic_least_incoming_arcs_first,
133     "3", eliminate_heuristic_contained_in_factor_first
134 ]
135 # Heuristic selection
136 elim_order = heuristic[ve.heuristic]
137 compare_with_ggpy = bool(input_user("Do you want to compare the result with ggpy? (yes/no): "))
138 # Run chosen algorithm
139 if algorithm_choice == "VE":
140     # Query is not None
141     ve.ve_marginal(query, evidence, elim_order)
142     if compare_with_ggpy:
143         ggpy_result = run_ggpy.ve_marginal(selected_bn_path, query, evidence)
144         # Print comparison after query
145         print("ggpy result:")
146         print(ggpy_result)
147     elif algorithm_choice == "MAP":
148         ve.ve_map(query_vars, evidence, elim_order)
149         if compare_with_ggpy:
150             ggpy_result = run_ggpy.map(selected_bn_path, query_vars, evidence)
151             # Print comparison after query
152             print("ggpy result:")
153             print(ggpy_result)
```

(b) run.py module - 2

Figure 1: run.py module

The main algorithm data structures for marginal probability distribution and MAP are initialized by creating an instance of the **VariableElimination** class. This object encapsulates the network representation and the methods needed to perform inference.

Subsequently, the user is asked to specify:

- The **network** i.e., the network represented as a .bif file to use;
- The **algorithm** to run
- The **query variable(s)**, i.e., the variable which posterior distribution will be computed in case of variable elimination execution, otherwise the list of variables to use as query to compute MAP;

- The **evidence**, represented as a (possibly empty) dictionary containing observed variables in the form `{variable: value}`.
- The **heuristic** to use to order the variables to perform the variable elimination

Input validation checks are performed to ensure the correctness of the inputs. Furthermore, the user is asked to specify whether he wants to compare the results obtained by the execution of the algorithm with those obtained using *pgmpy*, a benchmark library to perform inference.

Finally, the chosen algorithm's method is called, passing as parameters the query variable(s), the evidence dictionary, and the selected heuristic function used to determine the elimination order for the variable elimination process. The results will be eventually compared with those returned by *pgmpy*.

```

11 class VariableElimination():
12
13     def __init__(self, network, debug = False):
14         """
15         Initialize the variable elimination algorithm with the specified network.
16         """
17
18         self.debug = debug
19
20         # Network
21         self.network = network
22         # The nodes in the network
23         self.nodes = network.nodes
24         # The possible values of each node in the network
25         self.values = {}
26         # The cardinality of each node in the network
27         self.cardinalities = {}
28         # The parents of each node in the network
29         self.parents = {}
30         # The network probabilities table
31         self.probabilities = {}
32
33         # Initialize network information
34         for node in self.nodes:
35             self.values[node] = network.values[node]
36             self.cardinalities[node] = len(network.values[node])
37             self.parents[node] = network.parents[node]
38             self.probabilities[node] = network.probabilities[node]
39
40

```

Figure 2: VariableElimination Initialization

The constructor of the `VariableElimination` class is responsible for initializing all the data structures required to perform inference on a given Bayesian Network, as shown in Figure 2.

The class receives as input:

- a **network** object representing the network structure,
- a boolean flag **debug** used to enable detailed execution logs.

During this step, the structural information of the network is extracted and reorganized into internal data structures of the class.

Before describing the methods of the **VariableElimination** class, it is necessary to introduce the **Factor** class.

A **factor** is a data structure that encodes a function over a set of random variables. In our implementation, each instance of the **Factor** class is composed of:

- a list of variables involved in the factor,
- a list of cardinalities corresponding to each variable,
- a Conditional Probability Table (CPT) stored as a **pandas** DataFrame.

Furthermore, the **Factor** class implements operations required to perform Bayesian Network inference:

- **factor_reduction()**: restricts a factor given an observed variable (evidence application);
- **factor_product()**: computes the product between two factors, combining their variables and multiplying their probability values;
- **factor_marginalization()**: eliminates a variable by summing over its domain;
- **factor_maximizing()**: eliminates a variable by maximizing over its domain.

The first part of the assignment involves implementing the core operations required to manipulate factors in the context of Bayesian Networks. Below is a description of the developments.

The **factor_reduction()** method (Figure 3) reduce a factor given a variable and one of its values to the rows of the CPT consistent with the observed value. If the specified variable is not present in the factor, the method returns the factor unchanged. Otherwise, the CPT is filtered according to the selected value and the corresponding variable is removed from the list of variables and cardinalities, and a new reduced factor is returned.

```

25 def factor_reduction(self, variable, value):
26     """
27     Perform the factor reduction.
28
29     variable: the variable to reduce
30     value: the value of the variable to be reduced
31
32     """
33
34     # If the variable is not in the factor variables return the factor as is
35     if variable not in self.variables:
36         return self
37
38     new_cpt = self.cpt.copy()
39
40     # Filter cpt rows by value of variable
41     new_cpt = new_cpt[new_cpt[variable] == value]
42     # Remove variable column from cpt
43     new_cpt = new_cpt.drop(columns=[variable])
44
45     # Filter variables and cardinalities
46     new_vars = [var for var in self.variables if var != variable]
47     new_cards = [card for var, card in zip(self.variables, self.cardinalities) if var != variable]
48
49     # Order CPT columns with prob as last column
50     new_cpt = new_cpt[new_vars + ["prob"]]
51
52     # Return the new factor
53     return Factor(new_vars, new_cards, new_cpt)

```

Figure 3: factor_reduction method

The `factor_product()` method (Figure 4) computes the product of two factors. It identifies the variables shared by both factors and performs a merge operation on the corresponding CPTs. If no common variables exist, a cross product is performed. The resulting probabilities are obtained by multiplying the probability values of the two factors.

```

55 def factor_product(self, f2):
56     """
57     Perform the factor product.
58
59     f2: the second factor to multiply
60
61     """
62
63     # Filter the variables in common between the factors
64     common_vars = [var for var in self.variables if var in f2.variables]
65     # Create a list with the common variables of the factors
66     joined_vars = self.variables + [var for var in f2.variables if var not in common_vars]
67
68     # Create a list with the cardinalities of the joined variables
69     joined_cards = []
70
71     for var in joined_vars:
72         if var in self.variables:
73             var_index = self.variables.index(var)
74             joined_cards.append(self.cardinalities[var_index])
75         else:
76             var_index = f2.variables.index(var)
77             joined_cards.append(f2.cardinalities[var_index])
78
79     # If there are no variables in common just multiply the factors, otherwise merge the CPTs based on the common variables
80     if len(common_vars) == 0:
81         # Cross product
82         merged_cpt = self.cpt.merge(f2.cpt, how="cross")
83     else:
84         # Merge common vars
85         merged_cpt = self.cpt.merge(f2.cpt, on=common_vars)
86
87     # Calculate the combined probability of the common variables
88     merged_cpt["prob"] = merged_cpt["prob_x"] * merged_cpt["prob_y"]
89     # Remove the support columns used in the merge
90     merged_cpt = merged_cpt.drop(columns=["prob_x", "prob_y"])
91
92     # Order CPT columns with prob as last column
93     merged_cpt = merged_cpt[joined_vars + ["prob"]]
94
95     return Factor(joined_vars, joined_cards, merged_cpt)

```

Figure 4: factor_product method

The `factor_marginalization()` method (Figure 5) eliminates a variable from a factor by summing over its domain. If the variable is not present, the factor is returned unchanged. Otherwise, the CPT is grouped by the remaining variables and the probabilities are summed.

```

97 def factor_marginalization(self, var_to_sum):
98     """
99     Perform the factor marginalization.
100
101     var_to_sum: the variable to marginalize
102
103     """
104
105     # If the variable to marginalize is not in the factor variables return the factor as-is
106     if (var_to_sum not in self.variables):
107         return self
108
109     # If there is only 1 variable (and it is the one to marginalize) marginalize it
110     if (len(self.variables) == 1):
111         new_vars = []
112         new_cards = []
113         new_cpt = pd.DataFrame({"prob": [self.cpt["prob"].sum()]})
114         return Factor(new_vars, new_cards, new_cpt)
115
116     # Remove var to sum column
117     new_vars = [var for var in self.variables if var != var_to_sum]
118     # Set new_cards with the cardinalities of the new vars
119     new_cards = [card for var, card in zip(self.variables, self.cardinalities) if var != var_to_sum]
120     # Set new_cpt with the sum of the new_vars probabilities
121     new_cpt = self.cpt.groupby(new_vars, observed=False)["prob"].sum().reset_index()
122
123     # Order CPT columns with prob as last column
124     new_cpt = new_cpt[new_vars + ["prob"]]
125
126     return Factor(new_vars, new_cards, new_cpt)

```

Figure 5: factor_marginalization method

The `factor_maximizing()` method (Figure 6) eliminates a variable by taking the maximum probability over its domain instead of summing. In addition to the maximized factor, the method also returns a set of backtracking pointers, which store the value of the maximized variable corresponding to each configuration of the remaining variables. These pointers are later used during the MAP procedure to reconstruct the most probable assignment of the query variables.

```

107 def factor_maximizing(self, var_to_maximize, cpt):
108     """
109     Perform the factor maximizing.
110
111     var_to_maximize: the variable to maximize
112
113     """
114
115     # If the variable to maximize is not in the factor variables return the factor as-is
116     if (var_to_maximize not in self.variables):
117         return self, None
118
119     # If there is only 1 variable (and it is the one to maximize) maximize it
120     if (len(self.variables) == 1):
121         new_vars = []
122         new_cards = []
123         new_cpt = pd.DataFrame({"prob": [self.cpt["prob"].max()]})
124
125         # Backtrack CPT
126         index_max = self.cpt["prob"].idxmax()
127         max_val = self.cpt.loc[index_max, var_to_maximize]
128
129         backtrack = {
130             "max_val": max_val,
131             "pointer": [0] * len(self.variables)
132         }
133
134     return Factor(new_vars, new_cards, new_cpt), backtrack

```

(a) Initialization

```

125 new_vars = [var for var in self.variables if var != var_to_maximize]
126 # Set new_cards with the cardinalities of the new vars
127 new_cards = [card for var, card in zip(self.variables, self.cardinalities) if var != var_to_maximize]
128 # Set new_cpt with the max of the new_vars probabilities
129 new_cpt = self.cpt.groupby(new_vars, observed=False)["prob"].max().reset_index()
130
131 # Backtrack for backtracking
132 # Find max value
133 index_max_value = self.cpt.groupby(new_vars, observed=False)["prob"].idxmax()
134 # Select the row with max value (variables and prob)
135 row = self.cpt.loc[index_max_value]
136
137 # Create backtrack dictionary (dictionary/variables_assignment): values
138 max_val_max_pointer = {}
139
140 # For each row in new_cpt
141 for i, row in new_cpt.iterrows():
142     # Get the value for var in new_vars
143     value = row[var_to_maximize]
144     new_val_max_pointer[i] = value
145
146 # Order CPT columns with prob as last column
147 new_cpt = new_cpt[new_vars + ["prob"]]
148
149 # Backtrack CPT
150 backtrack = {}
151 backtrack["max_val"] = max_val
152 backtrack["pointer"] = new_val_max_pointer
153
154 return Factor(new_vars, new_cards, new_cpt), backtrack

```

(b) Return

Figure 6: factor_maximizing method

Before executing the Variable Elimination procedure, the Bayesian Network is converted into a set of factors. Each node is represented as a factor containing the variable of the node itself and its parents and the related CPT.

This conversion is performed by the `build_factors()` (Figure 7) method subject of the first part of the assignment. It iterates over all the nodes in the network and, for each node, constructs the corresponding factor by defining the list of variables as `[node] + parents`. It then extracts the associated cardinalities from the domain of each variable and copies the node's CPT from the

network representation to create a new `Factor` object.

```

42 def build_factors(self) -> list[Factor]:
43     """
44     Create the factor representation for each node in the network and return the factors list
45
46     Input:
47         None
48
49     Output:
50         The list of factors
51     """
52
53     factors: list[Factor] = []
54
55     for node in self.nodes:
56
57         # Set factor variables on node + its parent(s)
58         parents = self.parents[node]
59         factor_vars = [node] + parents
60         cardinalities = [self.cardinalities[var] for var in factor_vars]
61
62         # Create conditional probability table
63         cpt = self.probabilities[node].copy()
64
65         # Sort the column of the factor to order var, its parents, prob
66         cpt = cpt[factor_vars + ["prob"]]
67
68         # Create the new factor
69         new_factor = Factor(factor_vars, cardinalities, cpt)
70         # Add the new factor to the list of factors
71         factors.append(new_factor)
72
73     if self.debug:
74         print("-----")
75         print(f"Factor created for node {node}:")
76         print(new_factor)
77
78     return factors

```

Figure 7: `build_factors` method

2.3 Inference methods implementation

Marginal and MAP inference rely on the same core Variable Elimination procedure. For this reason, the common variable elimination phase was implemented in a dedicated method, `variable_elimination()`, while the final inference step is handled separately by `ve_marginal()` and `map_inference()`.

The `ve_marginal(query, observed, elim_order)` method implements the posterior distribution of a query variable given a set of observed variables 1.

Algorithm 1 Variable Elimination marginal

Require: Bayesian Network, query variable Q , evidence E , heuristic function

Ensure: Posterior distribution of Q

```
1: Identify the query variable
2: Identify the observed variables (evidence)
3: Build the initial set of factors from the network
4: Reduce all factors containing observed variables
5: Compute an elimination ordering using the chosen heuristic
6: for all variable  $Z$  in the elimination order do
7:   if  $Z$  is not the query variable and not observed then
8:     Multiply all factors containing  $Z$ 
9:     Sum out  $Z$  to obtain a new factor  $f_Z$ 
10:    Remove the multiplied factors from the list
11:    Add  $f_Z$  to the list of factors
12:   end if
13: end for
14: Multiply the remaining factors
15: Sum out all variables except the query variable
16: Normalize the result
17: return the final probability distribution
```

The method receives as input:

- the query variable;
- a dictionary of observed variables in the form `{variable: value}`;
- a heuristic function used to determine the elimination order.

Firstly, the method first calls `build_factors()` to generate the initial set of factors representing the Bayesian Network as seen previously. For each observed variable, `factor_reduction()` is applied to all factors containing it. This operation restricts the CPTs to rows consistent with the observed values and removes the observed variable from the affected factors.

Subsequently, an elimination order is computed using the provided heuristic function. The choice of elimination order affects the size of intermediate factors and therefore the computational efficiency of the algorithm.

At this point, for each variable Z in the elimination order (excluding the query and observed variables), the algorithm performs the following steps:

1. Multiply all factors containing Z into a single joint factor.
2. Marginalize Z from the joint factor by summing over its domain.
3. Replace the original factors the new factor

Finally, after all eliminations have been performed, the remaining factors are multiplied together and any remaining variables other than the query are summed out.

The resulting distribution is normalized to ensure that the probabilities sum to 1, producing posterior distribution.

The computational complexity of the Variable Elimination algorithm could depend on the order in which the variables are eliminated. In this implementation the algorithm has been tested using three different heuristic strategies to determine the elimination ordering:

```

2 def simple_sorting(net, query, evidence, factors=None):
3     elim_order = net.nodes
4     elim_order = [node for node in elim_order if node != query and node not in evidence ]
5
6     return elim_order
7
8 def least_incoming_arcs_first(net, query, evidence, factors=None):
9     # Remove query and evidence
10    elim_order = simple_sorting(net, query, evidence)
11
12    # Sort by number of parents (ascending)
13    elim_order.sort(key=lambda node: len(net.parents[node]))
14
15    return elim_order
16
17 def contained_in_fewer_factors_first(net, query, evidence, factors=None):
18    # Remove query and evidence
19    elim_order = simple_sorting(net, query, evidence)
20
21    # Sort by number of factors containing a variable (ascending)
22    elim_order.sort(key=lambda node: len([factor for factor in factors if node in factor.variables]))
23
24    return elim_order

```

Figure 8: Elimination ordering heuristics

The `simple_sorting` heuristic simply returns all variables except the query and the observed variables, preserving the original order in which they appear in the network.

The `least_incoming_arcs_first` heuristic eliminates variables in ascending order of the number of parents they have in the network.

The `contained_in_fewer_factors_first` heuristic eliminates variables that appear in fewer current factors.

In addition to standard marginal inference, the project extends the Variable Elimination framework to compute Maximum a Posteriori (MAP) queries through the `map_inference(query, observed, elim_order)` method. MAP inference shares the same initial steps as VE: the conversion of the Bayesian Network in factors, the reduction of the evidences and the elimination step performed on the variable elimination ordering computed using the chosen heuristic.

However, in this case, instead of marginalizing all non-query variables, MAP performs a maximizing step. After the variable elimination process the algorithm multiplies the remaining factors and applies the `factor_maximizing()` operation to the query variable. This produces a reduced factor containing the maximum probability for each configuration of the variables. Through the use of the backtracking pointers, which map each configuration of the remained vari-

ables to the value of the maximum probability, is then possible reconstructing the most probable assignment of the query variables to explain the observations.

Algorithm 2 MAP

Require: Bayesian Network, query variable Q , evidence E , heuristic function

Ensure: Most probable assignment of Q given E

- 1: Identify the query variable
 - 2: Identify the observed variables (evidence)
 - 3: Build the initial set of factors from the network
 - 4: Reduce all factors containing observed variables
 - 5: Compute an elimination ordering using the chosen heuristic
 - 6: **for all** variable Z in the elimination order **do**
 - 7: **if** Z is not the query variable and not observed **then**
 - 8: Multiply all factors containing Z
 - 9: Sum out Z to obtain a new factor f_Z
 - 10: Remove the multiplied factors from the list
 - 11: Add f_Z to the list of factors
 - 12: **end if**
 - 13: **end for**
 - 14: Multiply the remaining factors into a joint factor f
 - 15: Maximize over Q and store the backtracking pointers
 - 16: Reconstruct the most probable assignment using the pointers
 - 17: **return** MAP assignment for Q
-

From an operative point of view, both marginal inference (VE) and MAP inference share the same initial computational steps: factor construction, evidence application and elimination of non-query and non-evidence variables through the iterative elimination step. For that reason, to avoid redundancy and keep the implementation modular, the common steps were moved into a dedicated method `variable_elimination()`, which performs the elimination phase and returns the resulting reduced set of factors.

The inference algorithms diverge then only in the final operation:

- the `ve_marginal()` method computes the normalized posterior distribution by marginalizing the remaining variables;
- the `map_inference()` method applies maximization to the query variable and uses the associated backtracking pointers returned by `factor_maximizing()` to reconstruct the most probable assignment.

This updated structure avoids duplication, improves readability, and highlights the conceptual relationship between VE and MAP, which differ only in the final aggregation step applied after the elimination phase.

The shared procedure `variable_elimination()` showed in Figure 9a and 9b takes as input a query variable, a set of observed variables, and a heuristic function

used to compute the elimination order. It then builds the initial set of factors from the Bayesian Network, applies factor reduction on the observed variables, computes an elimination order, and iteratively eliminates the non-query and non-observed variables.

[illegible]

(a) variable_elimination() method - 1

```

15 if (self.factor_included == 0) {
16     # If we've already passed over other factors calculate the product of them
17     joint_factors = factor_included[]
18
19     for factor in factor_included[1:]:
20         joint_factors = joint_factors*factor_product(factor)
21
22     if self.debug:
23         print(joint_factors, ", joint_factor")
24
25     # Marginalize by summing over factor
26     new_factor = joint_factors_factor_marginalization(self.factor_included)
27     self.factor_included = new_factor
28     self.factor_not_included += [new_factor]
29
30     if self.debug:
31         print(".....")
32         print("Factor list after iteration %i" % iteration)
33         print(joint_factors)
34
35     if not ending_loop:
36         with self.lock:
37             # Factor iterations after elimination step: %i
38             for factor in self.factor_included:
39                 variables = self.joint_factor_variables
40                 self.factor_variables[factor] = variables
41                 self.factor_variables[factor] = self.factor_variables[factor].get('value', 0)
42                 self.factor_variables[factor] = self.factor_variables[factor].get('string', 'impossible')
43                 self.factor_variables[factor] = self.factor_variables[factor].get('value', 0)
44
45 return factors

```

(b) variable_elimination() method - 2

Figure 9: variable_elimination method

The result of the Variable Elimination phase is then used by the `ve_marginal()` method to compute the posterior distribution of the query variable by multiplying the remaining factors, marginalizing all non-query variables, and normalizing the final result, as shown in Figure 10.

```

150 def ve_marginal(
151     # 1. calculate result by doing the product of the remaining factors
152     result = factors[0]
153     for factor in factors[1:]:
154         result = result.factor_product(factor)
155
156     if self.debug:
157         print("-----")
158         print("Joint factor before marginalization:")
159         print(result.cpt)
160
161     with open(log_file, "a") as f:
162         f.write("\n\n- JOINT FACTOR BEFORE MARGINALIZATION:\n\n")
163         f.write(result.cpt.to_string(index=False))
164
165     # 2. Sum out all variables except the query
166     for var in list(result.variables):
167         if var != query:
168             result = result.factor_marginalization(var)
169
170     if self.debug:
171         print("-----")
172         print("Factor after marginalizing all non-query variables:")
173         print(result.cpt)
174
175     with open(log_file, "a") as f:
176         f.write("\n\n- FACTOR AFTER MARGINALIZING NON-QUERY VARIABLES:\n\n")
177         f.write(result.cpt.to_string(index=False))
178
179     # 3. normalize result
180     total = result.cpt["prob"].sum()
181     result.cpt["prob"] = result.cpt["prob"] / total
182
183     if self.debug:
184         print("-----")
185         print("Final normalized distribution:")
186         print(result.cpt)
187
188     with open(log_file, "a") as f:
189         f.write("\n\n- NORMALIZED RESULT:\n\n")
190         f.write(result.cpt.to_string(index=False))
191
192     return result

```

Figure 10: `ve_marginal()` method

At the same time, the result of the Variable Elimination phase is also used by the `map_inference()` method. In this case, the algorithm performs a maximization over the query variables. This is done by multiplying all factors containing the query variable, applying the `factor_maximizing()` operation to obtain the maximum probability, and storing backtracking pointers to reconstruct the most probable assignment. Finally, a traceback phase is performed to recover the MAP assignment for the query variables, as shown in Figure 11a and 11b.

```
def map_inference():
    # Find best assignment to the maximization ordering
    for y in max_ordering:
        if self.debug:
            print("Maximizing variable:", y)
        with open(log_file, "a") as f:
            f.write("Maximizing variable: (%s)" % y)
        # 1. Multiply all factors containing y
        factors_involved = [factor for factor in factors if y in factor.variables]
        factors_not_involved = [factor for factor in factors if y not in factor.variables]
        # 2. Find the best assignment for y
        if len(factors_involved) > 0:
            # Find the best assignment for y
            joint_factor = factors_involved[0]
            for factor in factors_involved[1:]:
                joint_factor = joint_factor * factor
            if self.debug:
                print("Joint factor before maximization")
                print(joint_factor.cpt)
            with open(log_file, "a") as f:
                f.write("Joint factor before maximization")
                f.write(joint_factor.cpt.to_string(indent=4))
            # 3. Find the best assignment for y
            final_factor, backtrack_pointer = joint_factor.factor_maximizing(y)
            if self.debug:
                print("Factor after maximizing over", y)
                print(final_factor.cpt)
                print("Backtracking pointer returned")
                print(backtrack_pointer)
            with open(log_file, "a") as f:
                f.write("Final factor after maximizing over the query (%s)" % y)
                f.write(joint_factor.cpt.to_string(indent=4))
                f.write("Backtracking pointer: " + str(backtrack_pointer))
        # 4. Store the best assignment and prob value
        backtrack_pointers[y] = backtrack_pointer
```

(a) `map_inference()` method - 1

```
factors = factors_not_involved + [final_factor]
# 5. Find the best assignment
best_query_assignment = {}
# 6. Trace back the best assignment to find the best values for the other variables
for y in reversed(max_ordering):
    backtrack = backtrack_pointers[y]
    conditional_vars = backtrack["vars"]
    pointer = backtrack["pointer"]
    key = tuple(best_query_assignment[var] for var in conditional_vars)
    best_query_assignment[y] = pointer[key]
# 7. Return the best assignment
best_query_assignment = {var: best_query_assignment[var] for var in max_ordering}
if self.debug:
    print("Best query assignment")
    print(best_query_assignment)
    print("Best query assignment")
with open(log_file, "a") as f:
    f.write("Final best assignment: (%s)" % best_query_assignment)
return best_query_assignment
```

(b) `map_inference()` method - 2

Figure 11: `factor_maximizing` method

2.4 Expectation Maximization algorithm implementation

The Expectation-Maximization (EM) algorithm is implemented to estimate the conditional probability tables (CPTs) of a Bayesian Network in the presence of hidden variables. The algorithm iteratively alternates between two steps: the Expectation step (E-step) and the Maximization step (M-step), until the convergence is achieved. The procedure starts from a random initialization of the parameters and then repeatedly updates them by computing the posterior distribution over the hidden variables and re-estimating the CPT parameters.

Expectation Maximization algorithm:

Initialize theta randomly

repeat until convergence:

E-step:

Compute $q(h) = P(H = h \mid E = e; \theta)$

Complete data using weighted samples $q(h)$

M-step:

Update theta using maximum likelihood estimation

For this part of the assignment, we introduced a dedicated main module named `run_em.py`, which loads the *Endorisk Bayesian Network* and the corresponding dataset, configures the EM experiment, executes the algorithm through the `run_expectation_maximization()` method, and stores the generated logs and plots used to track the behavior of the algorithm across different runs.

The `run_expectation_maximization()` (Figure 12) method controls the iterative learning process. The algorithm starts by setting the random seed and initializing the CPT θ parameters randomly for the nodes involving hidden variables. In contrast, for the nodes which don't involve hidden variables the CPTs are initialized using estimates calculated from the empirically observed data of the dataset. Subsequently, for each iteration, the EM algorithm alternates between the E-step and the M-step until convergence is reached.

After each iteration, the updated parameters are compared with those from the previous step by computing the maximum absolute difference (`max_diff`). This value is used as a convergence evaluation parameter. The process continues until the variation falls below the threshold. Finally, the learned CPTs and the convergence history are returned.

```
def evaluate_convergence(old_theta, new_theta) -> tuple(float, bool):
    max_diff = 0.0
    for node in self.vc.nodes:
        diff = (new_theta[node]['prob'] - old_theta[node]['prob']).abs().max()
        if diff > max_diff:
            max_diff = diff
    return max_diff, max_diff < convergence_threshold

# INITIALIZE & PROBABILITY
self.initialize_theta()
convergence_flag = False
iteration_count = 0
max_diff_history = []

# Repeat until convergence
while not convergence_flag:
    # Save old theta to compare
    old_theta = {node: self.vc.probabilities[node].copy() for node in self.vc.nodes}
    # Perform E-step: get expected counts
    expected_counts = self.e_step(data, hidden_vars, vc_elm_order)
    # Perform M-step: get new theta by performing e_step
    self.e_step(expected_counts)
    # Save new theta just a step
    new_theta = {node: self.vc.probabilities[node].copy() for node in self.vc.nodes}
    # Evaluate convergence
    max_diff, convergence_flag = evaluate_convergence(old_theta, new_theta)
    # Save max_diff history
    max_diff_history.append(max_diff)
```

(a) `map_inference()` method - 1

```
while not convergence_flag:
    # Save old theta to compare
    old_theta = {node: self.vc.probabilities[node].copy() for node in self.vc.nodes}
    # Perform E-step: get expected counts
    expected_counts = self.e_step(data, hidden_vars, vc_elm_order)
    # Perform M-step: get new theta by performing e_step
    self.e_step(expected_counts)
    # Save new theta just a step
    new_theta = {node: self.vc.probabilities[node].copy() for node in self.vc.nodes}
    # Evaluate convergence
    max_diff, convergence_flag = evaluate_convergence(old_theta, new_theta)
    # Save max_diff history
    max_diff_history.append(max_diff)
    if self.debug:
        print(f"Iter {iteration_count}, max_diff = {max_diff}")
    # Log each step
    self.log(self.log_file, "e") as f:
        f.write(f"\\n Iteration {iteration_count}, max_diff = {max_diff} \\n")
    iteration_count += 1
# Log final theta
self.log_theta("FINAL THETA")
return iteration_count, self.vc.probabilities, max_diff_history, max_diff_history[-1]
```

(b) `map_inference()` method - 2

Figure 12: `run_expectation_maximization()` method

Firstly, we perform the initialization step of θ , during this phase, the CPTs of the nodes involving the hidden variables are initialized randomly while for the other nodes we estimate the probabilities from the dataset. For nodes without parents, marginal probabilities are computed, while for nodes with parents, conditional probabilities are estimated for each parent configuration by counting occurrences in the data and normalizing them (Figure 13).

```

111 for node in nodes:
112     node_parents = parents[node]
113     cpt = self.ve.proBABILITIES[node].copy()
114
115     involve_hidden = (node in hidden_vars) or any(parent in hidden_vars for parent in node_parents)
116
117     # Step 1: Initialize hidden variables: initialize randomly
118     if involve_hidden:
119         # Node without parents
120         if len(node_parents) == 0:
121             random_values = []
122             sum_random_values = 0
123
124             # Sample random values
125             for v in values[node]:
126                 random_value = random.uniform(0, 1)
127                 random_values.append(random_value)
128                 sum_random_values += random_value
129
130             # Normalize
131             normalized_values = [v * sum_random_values for v in random_values]
132             cpt["prob"] = normalized_values
133
134         # Node with parents
135         parents_configurations = cpt.groupby(node_parents, observed=False)
136         conf_index = group_index
137
138         for parents_conf, group in parents_configurations:
139             random_values = []
140             sum_random_values = 0
141
142             # Generate random values for configurations
143             for v in range(len(group)):
144                 random_value = random.uniform(0, 1)
145                 random_values.append(random_value)
146                 sum_random_values += random_value
147
148             # Normalize
149             normalized_values = [v * sum_random_values for v in random_values]
150             cpt.loc[conf_index, "prob"] = normalized_values

```

(a) initialize_theta() method - 1

```

151 else:
152     # Step 2: CPT does not involve hidden variables: initialize from data
153
154     # Node without parents
155     if len(node_parents) == 0:
156         node_counts = data[node].value_counts(dropna=True)
157         total_rows = node_counts.sum()
158
159         for idx, row in cpt.iterrows():
160             node_value = row[node]
161             count_node = node_counts.get(node_value, 0)
162
163             if total_rows > 0:
164                 cpt.loc[idx, "prob"] = count_node / total_rows
165             else:
166                 cpt.loc[idx, "prob"] = 0.0
167
168     # Node with parents
169
170     counts_node_and_parents = data[node_parents + [node]].value_counts(dropna=True)
171     parent_counts = data[node_parents].value_counts(dropna=True)
172
173     for idx, row in cpt.iterrows():
174         # Build key: concatenate parent values and node value
175         parent_values = [row[parent] for parent in node_parents]
176         node_value = row[node]
177
178         parent_key = tuple(parent_values)
179         full_key = parent_key + [node_value]
180
181         parent_count = parent_counts.get(parent_key, 0)
182         count_node_and_parents = counts_node_and_parents.get(full_key, 0)
183
184         if parent_count > 0:
185             cpt.loc[idx, "prob"] = count_node_and_parents / parent_count
186         else:
187             cpt.loc[idx, "prob"] = 0.0
188
189     self.ve.proBABILITIES[node] = cpt
190
191 # Log the results
192 self.log_theta("INITIAL THETA")

```

(b) initialize_theta() method - 1

Figure 13: initialize_theta() method

The E-step is performed by the `e_step()` method, which computes the expected counts for each CPT entry by estimating the posterior distribution of the hidden variables for each data instance, as shown in Figure 14.

```

142 def e_step()
143     # Initialize expected counts for each node
144     expected_counts = {}
145     for node in self.ve.nodes:
146         expected_counts[node] = self.ve.proBABILITIES[node].copy()
147         expected_counts[node]["prob"] = 0.0
148
149     # Process each dataset row
150     for i, (_, data_row) in enumerate(data.iterrows()):
151         # Build observed evidence E = e (exclude hidden vars)
152         observed = {}
153         for var, value in data_row.items():
154             if var not in hidden_vars and pd.notna(value):
155                 observed[var] = value
156
157         # Compute posterior using hidden variables as query q(h) = P(H | observed; θ)
158         posterior = self.ve.distribution(hidden_vars, observed, ellm_order)
159         # Complement data with weighted samples q(h)
160         for _, row_assignment in posterior.cpt.iterrows():
161             complete_assignment = observed.copy()
162
163             for hidden_var in hidden_vars:
164                 complete_assignment[hidden_var] = row_assignment[hidden_var]
165
166             # Weight of this assignment = q(h)
167             weight = row_assignment["prob"]
168             for node in self.ve.nodes:
169                 # Take node CPT (node target + parents + prob)
170                 node_expected_counts = expected_counts[node]
171                 # Node complete assignment
172                 node_complete_assignment = complete_assignment[node]
173                 parents = self.ve.parents[node]
174
175                 # Find matching CPT row
176                 for row_index, cpt_row in node_expected_counts.iterrows():
177                     if cpt_row[node] != node_complete_assignment:
178                         continue
179
180                     match = True
181                     for parent in parents:
182                         if cpt_row[parent] != complete_assignment[parent]:
183                             match = False
184                             break
185
186                     # Update expected count with weight q(h)
187                     if match:
188                         node_expected_counts.loc[row_index, "prob"] += weight
189
190     return expected_counts

```

Figure 14: e_step() method

For each row of the dataset, the observed evidence is first extracted by excluding the hidden variables. Then, the method computes the posterior distribution

$$q(h) = P(H \mid E = e; \theta)$$

through the `ve_distribution()` method, which applies Variable Elimination to return the joint distribution over the hidden variables.

Each possible assignment of the hidden variables is then combined with the observed values to form a complete weighted sample. The weight associated with each completed assignment is given by its posterior probability $q(h)$. These weighted samples are finally used to update the expected counts of the corresponding CPT entries.

The M-step is performed by the `m_step()` method, which updates the CPT parameters as shown in Figure 15.

For each node, the method normalizes the expected counts to obtain the probability distributions. In the case of nodes without parents, normalization is applied over the entire CPT. For nodes with parents, normalization is performed separately for each parent configuration.

```

210 def m_step():
211     # Update H1 to the maximum likelihood estimator
212     total = 0
213
214     for node in self.ve.nodes:
215         cpt = expected_counts[node].copy()
216         parents = self.ve.parents[node]
217
218         # Case 1: node without parents
219         if (len(parents) == 0):
220             total = cpt['prob'].sum()
221
222             if total > 0:
223                 likelihood_estimator = cpt['prob']/total
224                 cpt['prob'] = likelihood_estimator
225             elif total == 0:
226                 cpt['prob'] = self.ve.proBABILITIES[node]['prob']
227
228         # Case 2: node with parents
229         else:
230             # Group CPT rows by parent configuration
231             group_cpt_by_parents = cpt.groupby(parents, observed=False)
232
233             for _, group_cpt in group_cpt_by_parents:
234                 index = group_cpt.index
235                 total = group_cpt['prob'].sum()
236
237                 if total > 0:
238                     # Normalize within the current parent configuration
239                     likelihood_estimator = group_cpt['prob']/total
240                     cpt.loc[index, 'prob'] = likelihood_estimator
241                 elif total == 0:
242                     # Keep previous probabilities if no expected counts are available
243                     cpt.loc[index, 'prob'] = self.ve.proBABILITIES[node].loc[index, 'prob']
244
245     # Store updated CPT
246     self.ve.proBABILITIES[node] = cpt

```

Figure 15: `m_step()` method

2.5 pgmpy integration for results validation

To validate the correctness of our implementation, we integrated the `pgmpy` library into the project using the following installation command:

```
pip install pgmpy
```

pgmpy is a Python library for causal and probabilistic modeling using graphical models. It provides a uniform API for building, learning, and analyzing models, such as Bayesian Networks. By integrating tools from both probabilistic inference and causal inference, pgmpy enables users to seamlessly transition between predictive and causal analyses [1].

In this project, pgmpy is used as a benchmark implementation to validate the correctness of the algorithms we developed. In particular, the results produced by our Variable Elimination and MAP inference implementations are compared with those returned by pgmpy for the same Bayesian networks, queries, and evidence variables.

To simplify the interaction with the library, we introduced a dedicated module named pgmpy_utils.py, shown in Figure 16. This module contains helper functions used to perform inference through the pgmpy framework.

```

1  ✓ from pgmpy.readwrite import BIFReader
2  ✓ from pgmpy.inference import VariableElimination
3
4  ✓ def load_pgmpy_model(bn_path: str):
5      reader = BIFReader(bn_path)
6      model = reader.get_model()
7      return model
8
9  ✓ def run_pgmpy_ve_marginal(bn_path: str, query: str, evidence: dict):
10     model = load_pgmpy_model(bn_path)
11     inference = VariableElimination(model)
12
13     ✓ ve_marginal_result = inference.query(
14         variables=[query],
15         evidence=evidence if evidence else None,
16         show_progress=False
17     )
18
19     return ve_marginal_result
20
21 ✓ def run_pgmpy_map(bn_path: str, query_vars: str, evidence: dict):
22     model = load_pgmpy_model(bn_path)
23     inference = VariableElimination(model)
24
25     ✓ map_inference_result = inference.map_query(
26         variables=query_vars,
27         evidence=evidence if evidence else None,
28         show_progress=False
29     )
30
31     return map_inference_result

```

Figure 16: Utility module for pgmpy-based inference

The function `load_pgmpy_model` takes as input the path to a Bayesian network encoded in `.bif` format and uses the `BIFReader` utility provided by pgmpy to construct the corresponding network model.

This function is internally used by two additional helper methods: `run_pgmpy_ve_marginal` and `run_pgmpy_map`, which perform marginal inference and MAP inference, respectively. As in our implementation, both procedures first build the Bayesian Network model from the `.bif` configuration file and then perform inference

using the Variable Elimination algorithm implemented in `pgmpy`.

In particular, marginal inference is performed through the `inference.query()` method, which returns the posterior probability distribution of the query variables given the observed evidence whereas MAP inference is computed using the `inference.map_query()` method, which returns the most probable assignment of the query variables given the evidence.

The outputs produced by these functions are used as a reference to verify that the results returned by our implementation are consistent with those obtained from the `pgmpy` library. This comparison provides an external validation of our implementation, ensuring that both inference procedures produce consistent and reliable results.

2.6 Testing Methodology

We will conduct several tests on our implementation in order to evaluate both correctness and computational efficiency.

First, we will perform a set of inference tests aimed at verifying that the results produced by our implementation are consistent. To evaluate that we will compare them with those obtained using the `pgmpy` library.

We begin by testing the Variable Elimination algorithm for marginal probability distribution calculation. For this purpose, we consider the “Burglary–Earthquake–Alarm–JohnCalls–MaryCalls” Bayesian Network, shown in Figure 17. We evaluate the algorithm under different inference configurations, including different query variables, single-evidence settings, and multiple-evidence settings.

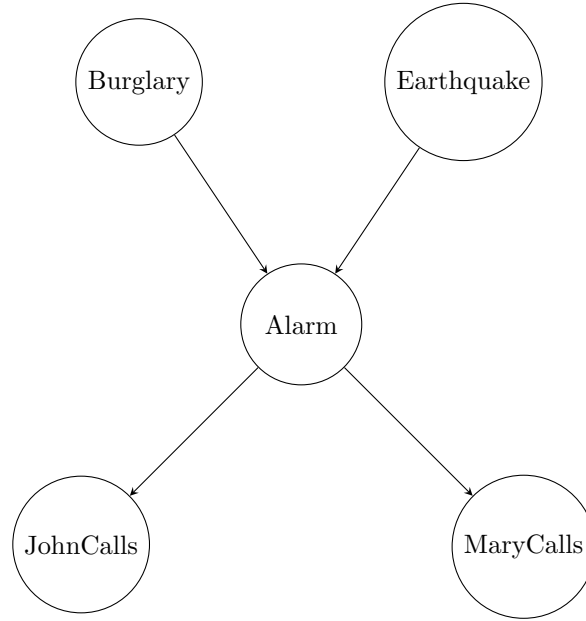


Figure 17: Bayesian Network structure of the Earthquake example.

Subsequently, we will evaluate the MAP inference procedure. In this case, we will test both single-variable MAP queries and multi-variable MAP queries, again comparing the returned assignments with those produced by `pgmpy`.

For the last part of the assignment we will focus on the testing of the Expectation-Maximization (EM) algorithm for learning the probability tables (CPTs) of a Bayesian Network with hidden variables. In particular, we analyze its behavior under different random initialization of the parameters. The experiments are conducted on the Endorisk Bayesian Network, where the variables *Lymph node metastasis* and *Myometrical invasion* are treated as hidden during the learning process.

Due to the huge computational cost of performing inference for each data row at every EM iteration, we do not execute the algorithm over the full dataset of approximately 10.000 instances. Instead, a subset of the data is used to run multiple tests within a reasonable time.

We start with a preliminary test in which we run the algorithm on a small subset of 50 samples to verify the correctness of implementation, outputs and logs and ensure that the algorithm converges as expected in suitable time. After this validation phase, the main experiments are conducted using a larger subset of the dataset.

Subsequently, we will execute the algorithm five times with different random seeds in order to evaluate the effect of the initialization on the EM algorithm. As

convergence threshold we use $\epsilon = 2e-2$ meaning a threshold of 0.02. We choose this value as a good trade-off between computational cost and convergence precision, allowing the algorithm to converge and terminate when parameters are sufficiently precise while keeping execution time manageable.

Finally, in addition to the convergence experiments, a final EM run will be performed over the full dataset to obtain a more reliable parameter estimate for comparison with the original *Endorisk model*. In this case, a less strict convergence threshold of 0.05 will be used again to keep the computational cost manageable.

3 Results

This section presents the results obtained from the tests described in the previous section.

The goal of the first test is to assess the correctness of our implementation by comparing its outputs with those produced by the `pgmpy` library. We evaluate the Variable Elimination algorithm for marginal probability distribution calculation on the “Burglary–Earthquake–Alarm–JohnCalls–MaryCalls” Bayesian Network. Several inference queries are executed, including marginal probability queries, queries with a single evidence variable, and queries with multiple evidence variables. For each configuration, we compute the resulting probability distribution using our implementation and compare it with the distribution returned by `pgmpy`.

Query	Evidence	Our output	pgmpy output
$P(\text{Burglary})$	–	True = 0.01 False = 0.99	True = 0.0100 False = 0.9900
$P(\text{Alarm})$	–	True = 0.016114 False = 0.983886	True = 0.0161 False = 0.9839
$P(\text{Burglary} \mid \text{Alarm} = \text{True})$	$\text{Alarm} = \text{True}$	True = 0.583461 False = 0.416539	True = 0.5835 False = 0.4165
$P(\text{Earthquake} \mid \text{JohnCalls} = \text{True})$	$\text{JohnCalls} = \text{True}$	True = 0.094858, False = 0.905142	True = 0.0949 False = 0.9051
$P(\text{Burglary} \mid \text{JohnCalls} = \text{True}, \text{MaryCalls} = \text{True})$	$\text{JohnCalls} = \text{True}$ $\text{MaryCalls} = \text{True}$	True = 0.556522 False = 0.443478	True = 0.5565 False = 0.4435

Table 1: Comparison between our Variable Elimination implementation and `pgmpy` results

The second test focuses on evaluating the correctness of the MAP inference algorithm. Similarly to the previous test, we compare the results produced by

our implementation with whose obtained using pgmpy.

In this case, the goal is to verify that the algorithm correctly identifies the most probable assignment of the query variables given the provided evidence. We consider different configurations, including MAP queries involving a single variable and queries involving multiple variables simultaneously. For each query, we compute the MAP assignment using our implementation and verify that it matches the assignment returned by pgmpy.

MAP Query	Evidence	Our output	pgmpy output
<i>Burglary</i>	–	Burglary=False	Burglary=False
<i>Alarm</i>	–	Alarm=False	Alarm=False
<i>Burglary</i>	<i>JohnCalls = True</i>	Burglary=False	Burglary=False
<i>Burglary, Earthquake</i>	<i>Alarm = True</i>	Burglary=True, Earthquake=False	Burglary=True, Earthquake=False
<i>Burglary, Earthquake</i>	<i>JohnCalls = True</i> <i>MaryCalls = True</i>	Burglary=True, Earthquake=False	Burglary=True, Earthquake=False

Table 2: Comparison between our MAP inference implementation and pgmpy results

Subsequently, we test the same queries using the three different heuristics for ordering the variables to perform VE presented in the Methods section and all the heuristics have been proved effective in correctly calculating the posterior probability distribution.

Finally, we perform the tests on our EM algorithm implementation. Below the results of the preliminary test of the EM algorithm with a sample of 50 rows and a convergence threshold of 0.05.

Seed	Sample Size	ϵ	Iterations	Final Max Diff
1	50	0.05	10	0.0463

Table 3: EM algorithm preliminary test results

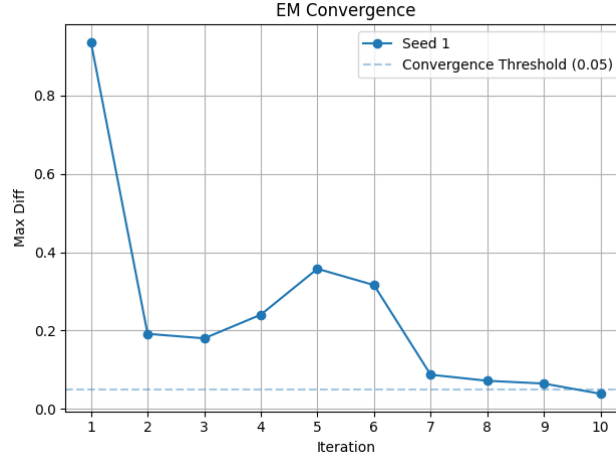


Figure 18: EM convergence preliminary test

At this point we are ready to proceed with the test of our EM implementation with a bigger sample of 100 records, a convergence threshold of 0.02 and 5 different random initializations in order to compare and evaluate how the initialization effect the execution of EM. Below the results of this experiments.

Seed	Sample Size	ϵ	Iterations	Final Max Diff
1	100	0.02	18	0.0192
2	100	0.02	33	0.0198
3	100	0.02	30	0.0169
4	100	0.02	29	0.0189
5	100	0.02	16	0.0152

Table 4: Number of iterations and final maximum parameter variation for each EM run under different random initializations.

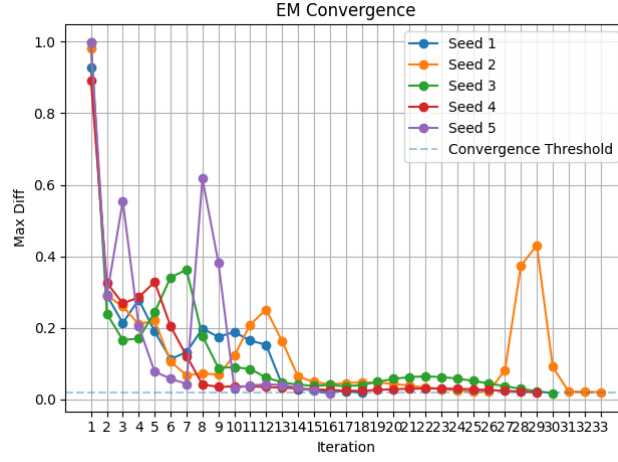


Figure 19: EM convergence with random initializations

Finally, here we show the results of the last test conducted using the full dataset and the comparison of the CPT of a node involving hidden variables between the BN model and the learned model after EM execution.

Seed	Sample Size	ϵ	Iterations	Final Max Diff
5	10000	0.05	5	0.046

Table 5: EM algorithm test on full dataset

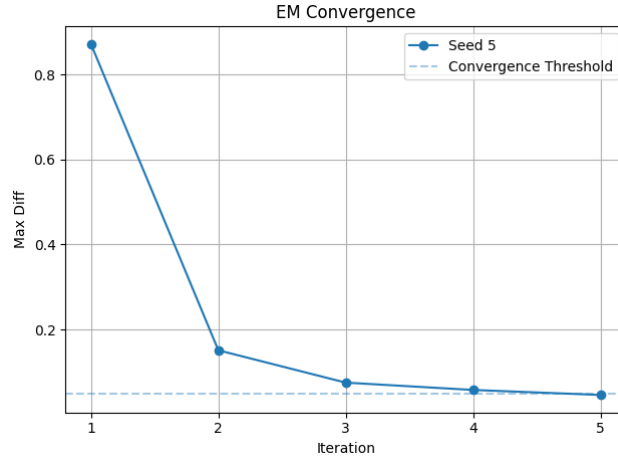


Figure 20: EM convergence - test on full dataset

LNM	Platelets	Original	Learned (EM)
no	lt_400	0.9698	0.9916
no	ge_400	0.0302	0.0084
yes	lt_400	0.9412	0.6731
yes	ge_400	0.0588	0.3269

Table 6: Comparison between original and learned CPT for Platelets given LNM

4 Discussion

In this section, we discuss the results obtained from the experimental evaluation of our implementation. First, the results of our inference algorithms for marginal posterior probability and MAP are consistent with those produced using *pgmpy* proving the correctness and reliability of our implementation.

In the final part, we analyzed the behavior of the EM algorithm. The preliminary test on a subset of 50 samples showed that the iteration between E-step and M-step has successfully allowed the algorithm to converge under the chosen threshold, indicating that the parameters are correctly updated through the execution. Figure 18 illustrates the evolution of the `max.diff` parameter used to evaluate convergence. The values initially fluctuate, then progressively stabilize, and finally fall below the threshold after by the 10th iteration.

The experiments conducted on a larger subset of 100 samples with five different random initializations provide the most interesting insights. In all runs, despite some fluctuations, the `max.diff` parameter shows an overall decreasing trend, eventually reaching convergence in every case. However, the number of iterations required to converge varies significantly across runs. For instance, the configuration with seed 5 converges after just 16 iterations, while seed 2 requires 33 iterations, more than the double. This highlights the sensitivity of the EM algorithm to the initial parameter values, as different random initializations can lead to different convergence behaviors.

Overall, the results confirm that the EM implementation is stable and converges consistently, while also highlighting the influence of random initialization on convergence speed.

Additionally, the test conducted on the full dataset highlights how, with a larger amount of data, the parameters tend to converge more rapidly below the threshold and with less oscillations. This suggests more data availability leads to more stable parameter updates. Furthermore, the comparison between the original model probabilities and those learned through the EM algorithm execution shows that the learned CPTs provide a reasonable approximation of the reference Bayesian Network with hidden variables. While the estimates are generally close for configurations involving fully observed variables, larger discrepancies are observed in the presence of hidden variables, where there is more uncertainty.

The comparison between the original and learned CPT for the variable Platelets shows that the learned model approximates the original distribution reasonably well for LNM = no, with only minor deviations. However, larger discrepancies are observed for LNM = yes, where the learned probabilities are more balanced compared to the original ones.

This behavior can be explained by the presence of hidden variables and the estimation process of the EM algorithm, which relies on inferred values rather than directly observed data. As a result, parameter estimates for configurations involving hidden variables tend to be more sensitive and may deviate from the original model, even when a large dataset is used.

The comparison between the original and learned CPT for the variable Platelets shows that the learned model approximates the original distribution reasonably well for LNM = no, with only minor deviations. However, larger discrepancies are observed for LNM = yes, where the learned probabilities are more balanced compared to the original ones. This behavior can be explained by the presence of hidden variables and the estimation process of the EM algorithm, which relies on inferred values rather than directly observed data. As a result, parameter estimates for configurations involving hidden variables tend to be more sensitive and may deviate from the original model, even when a large dataset is used.

The possible future extension of this work would be defining a quantitative measure of computational complexity for the Variable Elimination algorithm and using it to compare the impact of different elimination heuristics on the overall complexity with different networks. In this way we could be able to define the best way to save computational resources for the variable elimination process that, as we seen previously, is an essential part of the implementation of marginal and MAP inference methods and EM algorithm.

5 Resources

References

- [1] Ankur Ankan and Johannes Textor. “pgmpy: A Python Toolkit for Bayesian Networks”. In: *Journal of Machine Learning Research* 25.265 (2024), pp. 1–8. URL: <http://jmlr.org/papers/v25/23-0487.html>.
- [2] Fabrizio Ruggeri, Ron S. Kenett, and Frederick W. Faltin, eds. *Encyclopedia of Statistics in Quality and Reliability*. 1st ed. Wiley, 2007. ISBN: 978-0-470-01861-3. DOI: 10.1002/9780470061572.eqr089.